PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering
Software Engineering (UE24CS341A) — Level 3 Mini-Project | Semester 5, Section G | AY 2026-27

1.1 Project Title and Group

Field	Entry
Project Title	StudentRMS – Student Record Management System
Group Number	Team 4
Project ID	SRMS-2026-04
Technology	(console application, file-based persistence)
Complexity	Easy
Customer / Stakeholder	Department Academic Office (record-keeping staff)

1.2 Team Profile and Product Ownership

To strictly align with the requirement that each member must own at least one complete functional feature, the four functional areas of StudentRMS are split equally below. Each member owns one feature end-to-end and is individually responsible for the matching sections of both the Synopsis and the SRS.

Member	SRN	Owned Functional Feature	Synopsis Deliverables	SRS Deliverables
Member 1 — Samiksha Jayanth	**PES1UG24CS414**	**Administrator Authentication & Session Management**	**Synopsis Deliverables:** ● Problem Statement & Objectives for Authentication ● Executive Summary / Abstract	**SRS Deliverables:** ● Section 2.2: Purpose (Introduction) ● Section 2.4: User Classes & Characteristics ● Section 2.7: Security Requirements (SEC-001, SEC-002, SEC-005) ● FRs: FR-001 (Login/ Authentication) ● Use Cases: UC-01 (Administrator Login)
Member 2 — Rohan M	**PES1UG24CS381**	**Record Search, View & Listing**	**Synopsis Deliverables:** ● Problem Statement & Objectives for Search/Retrieval ● High-Level Scope & Project Boundaries	**SRS Deliverables:** ● Section 2.3: Scope (In Scope / Out of Scope) ● Section 2.6: NFR-001 (Performance) ● FRs: FR-004 (Search), FR-007 (List & Sort) ● Use Cases: UC-03 (Search/ View Record), UC-06 (List & Sort Records)

Member	SRN	Owned Functional Feature	Synopsis Deliverables	SRS Deliverables
Member 3 — Saksham Bajpayi	**PES1UG24CS404**	**Record Lifecycle Management (Add, Update, Delete)**	**Synopsis Deliverables:** ● Problem Statement & Objectives for Record Management ● Key Deliverables & Expected Outcomes	**SRS Deliverables:** ● Section 2.5: Functional Requirements (detailed FRs) ● Section 2.6: NFR-002 (Reliability) ● FRs: FR-002 (Add), FR-003 (Duplicate Check), FR-005 (Update), FR-006 (Delete), FR-012 (Validation) ● Use Cases: UC-02 (Add Record), UC-04 (Update Record), UC-05 (Delete Record)
Member 4 — Riddhi Mundada		**Reporting, CGPA Computation & Data Persistence**	**Synopsis Deliverables:** ● Problem Statement & Objectives for Reporting/ Persistence ● Target Audience & System Constraints	**SRS Deliverables:** ● Section 2.2: Design & Implementation Constraints (file-based storage) ● Section 2.12: Tool Stack ● FRs: FR-008 (CGPA Calc), FR-009 (Rank List), FR-010 (Persistence), FR-011 (Export) ● Use Cases: UC-07 (Generate Rank List / Report)

Important: UI design, file-handling support and unit testing are supporting responsibilities, not substitutes for owning a functional feature, per the project guidelines.

DELIVERABLE 2 – REQUIREMENTS

This package contains the Software Requirements Specification (SRS), initial validation planning and the project plan, organized into the same structure as the university-supplied SRS template: introduction, overall description, interfaces, analysis models, system features, non-functional requirements, business rules, glossary, field layouts and a requirement traceability matrix.

2.1 SRS – Overview / Required Structure

Section	What This Package Covers	StudentRMS Example
1. Introduction	Purpose, audience, scope, references	Why the system exists and who reads the SRS
2. Overall Description	Perspective, functions, users, environment, constraints, assumptions	Administrator/console environment, single-user
3. External Interfaces	UI, software and communication interfaces	Console menu, local file I/O
4. Analysis Models	Use case and other relevant models	Use case diagram + record data model
5. System Features	Feature description, stimulus/response, detailed FRs	Add/Search/Update/Delete Record, Report
6. Nonfunctional Requirements	Performance, safety, security, quality attributes, business rules	Response time, authentication, reliability
7. Other Requirements	Database/legal/localization/reuse/etc.	File-format and data-retention requirements
Appendix A	Glossary	CGPA, Record, Administrator
Appendix B	Field layouts/report requirements	struct Student fields

Section	What This Package Covers	StudentRMS Example
Appendix C	Requirement Traceability Matrix	REQ → architecture → design → code → tests

2.2 Purpose

The purpose of StudentRMS is to provide a `C11` console-based platform for an academic-office administrator to add, search, update, delete, sort and report on student records, with automatic percentage/CGPA computation and persistent file-based storage.

2.3 Scope

In Scope	Out of Scope
Administrator authentication	Multi-user role hierarchy (only one administrator role)
Add / search / update / delete student records	Student self-service portal / GUI
Automatic percentage / CGPA calculation	Networked / multi-client access
Sorting and rank-list generation	Integration with a university ERP system
File-based persistent storage	Relational database (DBMS) backend
Text-file report export	Web or mobile interface

2.4 User Classes

User Class	Privileges	Characteristics
Administrator	Full CRUD on student records, generate reports	Single trusted user; comfortable with a text-menu interface

2.5 Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall require an administrator to log in with a username and password before any record-management function is accessible.	High	Functional test
FR-002	The system shall allow an authenticated administrator to add a new student record containing roll number, name, department, semester and subject-wise marks.	High	Functional test
FR-003	The system shall reject an add-record request if the roll number already exists in the data file.	High	Negative test
FR-004	The system shall allow the administrator to search for a student record by roll number or by name.	High	Functional test
FR-005	The system shall allow the administrator to update the fields of an existing student record identified by roll number.	High	Functional test
FR-006	The system shall allow the administrator to delete an existing student record by roll number, after an explicit confirmation step.	High	Functional test
FR-007	The system shall display all student records sorted by roll number, name, or CGPA, as selected by the administrator.	Medium	System test
FR-008	The system shall automatically compute each student's percentage and CGPA from the stored subject marks.	High	Unit test
FR-009	The system shall generate a class rank list ordered by CGPA in descending order.	Medium	System test

ID	Requirement	Priority	Verification
FR-010	The system shall persist all student records to a data file such that records are retained after the program exits and are reloaded on the next run.	High	Integration test
FR-011	The system shall allow the administrator to export the current record list / rank report to a text file.	Medium	Functional test
FR-012	The system shall validate that mandatory fields are non-empty and that marks entered are within the range 0–100 before saving a record.	High	Negative + positive test

Requirement-writing rule: each requirement is uniquely identified and is concise, complete, unambiguous, verifiable and necessary, per the supplied SRS template.

2.6 Non-Functional Requirements

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	The system shall return the result of a search/list operation for up to 1000 records within 2 seconds on the reference test machine.	Performance test
NFR-002	Reliability	The system shall not corrupt or lose existing records if the program terminates unexpectedly during a write operation.	Fault-injection test
NFR-003	Usability	An administrator shall be able to complete any core operation (add/ search/update/delete) within 3 menu selections from the main menu.	Usability walkthrough
NFR-004	Maintainability	Business logic shall be separated into per-module source/header files with no function exceeding ~60 lines.	Code review
NFR-005	Testability	Each core module shall expose functions that can be exercised by a unit-test harness without requiring the interactive console menu.	Unit/integration test
NFR-006	Portability	The system shall compile and run without modification on both Linux (gcc) and Windows (MinGW) using only the C standard library.	Build verification

2.7 Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	The administrator password shall not be stored or compared as plaintext embedded in source code; it shall be read from a separate, restricted-access credentials file.	Code/configuration review
SEC-002	No record-management function shall be reachable from the console menu prior to successful administrator authentication.	Authorization test
SEC-003	All user-supplied input (roll number, name, marks) shall be length-checked and type-validated before use, to prevent buffer overflows in fixed-size C buffers.	Negative/security test
SEC-004	File read/write operations shall check return values and handle I/O errors gracefully without crashing or leaving the data file in a partially written state.	Fault-injection test
SEC-005	Failed login attempts shall be logged with a timestamp to a local log file.	System test

2.8 Business Rules / Domain Rules

- Only an authenticated administrator may add, update, or delete a student record.
- A roll number must be unique across all records.
- Marks for any subject must lie between 0 and 100 inclusive.
- Percentage and CGPA are always system-computed, never directly entered by the administrator.
- A record deletion requires explicit confirmation and cannot be undone.

2.9 Use Cases

Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Administrator Login	Administrator	Application launched	Authenticated session started
UC-02	Add Student Record	Administrator	Logged in	New record saved to file
UC-03	Search / View Record	Administrator	Logged in; records exist	Matching record displayed
UC-04	Update Student Record	Administrator	Record exists	Record updated in file
UC-05	Delete Student Record	Administrator	Record exists	Record removed from file
UC-06	List & Sort Records	Administrator	Logged in	Sorted list displayed
UC-07	Generate Rank List / Report	Administrator	Records exist	Report displayed and/or exported

2.10 Validation Specification – Initial Planning

At the requirements stage, the validation specification is planning-focused. This stage is not the final test report; detailed test cases and pass/fail evidence are produced in the Implementation & Testing and Final Report deliverables.

Requirement Category	Planned Validation	Example
Functional	Black-box functional tests on each menu operation	Add/search/update/delete a record with valid and invalid data
Security	Authentication bypass attempts; buffer-overflow/oversized-input tests	Attempt record operations without login; enter an over-length name
Performance	Timed list/search operation on a data file with ~1000 records	Measure response time against NFR-001
Usability	Task-based walkthrough with a first-time user	New user adds and retrieves a record unaided
Reliability	Interrupt the program mid-write and restart	Verify data file is not corrupted (NFR-002)
Unit	Automated unit tests + coverage on core functions	CGPA calculation, sort routine, validation functions

2.11 Project Plan

The project-plan guidance asks the team to identify and justify the lifecycle, identify tools, create a detailed WBS, and estimate effort in person-days.

Work Package	Tasks	Owner	Estimated Effort (person-days)	Output
WP1	Requirements + use cases	All	3	SRS
WP2	Architecture & module-level design	All	2	Design document
WP3	Admin authentication & session management	M1 (Samiksha)	4	Working feature
WP4	Search, view & listing module	M2 (Rohan)	4	Working feature
WP5	Record lifecycle: add / update / delete	M3 (Saksham)	6	Working feature
WP6	CGPA computation, ranking, reporting & persistence	M4 (Riddhi)	6	Working feature
WP7	Testing + CI/CD + security validation	All	4	Test/validation evidence
WP8	Final validation + report	All	3	Final report

Total estimated effort: 32 person-days across a 6-sprint, ~6-week schedule, tracked on the GitHub Projects board with weekly evaluation checkpoints.

2.12 Tool Stack

Activity	Recommended/Example Tool	Evidence to Submit
Version control	Git + GitHub	Repository URL, commit history
Agile planning	GitHub Projects / Jira	Backlog, user stories, sprint board
CI/CD	GitHub Actions	Pipeline run logs
Build system	GCC / Makefile	Build logs
Testing	Custom assert-based harness / Unity	Test results
Static analysis	cppcheck	Analysis report
Design	Draw.io / PlantUML	UML diagrams

Appendix A: Glossary

Term	Definition
CGPA	Cumulative Grade Point Average, computed from subject marks.
Record	A single student's stored data (roll number, name, marks, etc.).
Administrator	The single authenticated user permitted to manage records.
Data file	The local file used for persistent storage of all records.

Appendix B: Field Layouts (struct Student / credentials)

Field	Length	Data Type	Description	Mandatory
roll_no	10	char[]	Unique student roll number	Y
name	50	char[]	Student full name	Y
department	20	char[]	Department / branch code	Y
semester	2	int	Current semester (1–8)	Y
subject_marks[]	6 x float	float[]	Marks per subject (0–100)	Y
percentage	-	float (computed)	Derived percentage	N (auto)
cgpa	-	float (computed)	Derived CGPA	N (auto)
admin_username	20	char[]	Administrator login ID	Y
admin_password_hash	64	char[]	Stored credential (not plaintext)	Y

Appendix C: Requirement Traceability Matrix (Initial)

Architecture, design, code-file and test-case references will be filled in as the Design and Implementation deliverables are completed; requirement IDs and descriptions are fixed at this stage.

Sl. No	Req ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID
1	FR-001	Administrator login before access					

Sl. No	Req ID	Brief Description	Architecture Ref	Design Ref	Code File Ref	Test Case ID	System Test ID
2	FR-002	Add student record					
3	FR-004	Search record by roll no./name					
4	FR-005	Update student record					
5	FR-006	Delete student record					
6	FR-008	Automatic CGPA computation					
7	FR-010	Persist records to file					
8	SEC-003	Input length/type validation					

DELIVERABLE 2 — SRS COMPLETION ADDENDUM

StudentRMS — Student Record Management System | Team 4 | UE24CS341A

This addendum supplements the seven-page SRS. It completes the minimum functional-requirement and security-objective counts and corrects the data-model description to match the implementation.

1. Additional functional requirements

ID	Requirement	Priority	Acceptance / verification
FR-013	The system shall allow an authenticated administrator to view the local authentication audit log from the main menu.	Medium	After login, select menu option 11 and verify timestamped authentication events are displayed.
FR-014	The system shall allow the authenticated administrator to log out, clear authenticated session state, and terminate the application.	High	Select menu option 12; verify logout event is logged and the application exits.
FR-015	The system shall stop accepting login attempts after three consecutive failed administrator login attempts during startup and exit without displaying the main menu.	High	Enter three invalid credential pairs; verify lockout message, non-zero exit, and no main menu.

2. Security objectives

ID	Objective	Related requirements	Verification
SO-01 — Access control	Prevent unauthenticated users from performing student record-management operations.	FR-001, SEC-002	Attempt record operations without a successful authenticated session; verify access is denied.
SO-02 — Record integrity and auditability	Protect record integrity through validation and safe file handling, and maintain timestamped authentication events without logging passwords.	FR-012, SEC-003, SEC-004, SEC-005	Negative input tests, storage failure/corruption tests, audit-log review, and password-absence check.

3. SEC-001 accepted demonstration-scope deviation

The original SEC-001 wording requires credentials to be read from a separate restricted-access file and not embedded in source code. The project owner has accepted the compiled demonstration credentials (**admin / admin123**) for this academic project. This is an explicit scope deviation, not technical compliance with the original separate-file requirement and not a production security recommendation. Use synthetic academic data only. The other security requirements remain in scope and must be verified.

4. Corrected data model

The Student structure contains only student-record fields. The implementation in include/student.h defines roll_no as char[16], name as char[50], department as char[20], semester as int (1–8), subject_marks as float[6], percentage as a derived float, and cgpa as a derived float. There are no admin_username or admin_password_hash fields in Student. The demonstration credential is compiled in src/auth.c; no credentials.dat file is used.

5. Requirement traceability additions

Requirement	Architecture / module	Design / code reference	Test mapping
FR-013 / SEC-005	Authentication + Console UI	ui_view_audit_log(); log_auth_event()	Audit menu CI smoke test; verify timestamped events and no password in login.log.
FR-014	Authentication + Console UI	auth_logout(); auth_is_authenticated()	Logout/session-reset unit test and menu option 12 CI smoke test.
FR-015 / SEC-002	Authentication + Console UI	Three-attempt startup loop in main()	Three-invalid-attempt CI smoke test; confirm no main menu.
SO-01	Authentication + Console UI	Session gate before record operations	Authorization tests.
SO-02	Validation + Storage + Authentication	Field validation, storage status handling, timestamped logging	Input, storage, corruption and audit tests.

6. Acceptance status

The SRS now has 15 functional requirements (FR-001 to FR-015), at least five non-functional requirements (NFR-001 to NFR-006 in the source SRS), two explicit security objectives (SO-01 and SO-02), and five security requirements (SEC-001 to

SEC-005). SEC-001 remains an accepted academic-demo deviation from the original separate-file wording.